

CHAPTER FIVE: SUMMARY, CONCLUSION, AND RECOMMENDATIONS

5.1 Summary

This project set out to replace the manual, queue-based way food is ordered from campus vendors at University of Africa, Toru-Orua with a web-based system covering three roles: a student who browses vendors and places orders, a vendor who receives and fulfils those orders, and an administrator who controls which vendors are allowed to trade on the platform. Chapter Three analysed the existing manual process, identified six specific problems with it, and set out the requirements, architecture, database design and UML models that were used to guide the build. Chapter Four described the resulting system in detail: an Express/EJS server-rendered application with session-based authentication, a JSON-file persistence layer, a client-side cart and four-second status polling in place of WebSockets, a full dark-themed interface redesign built on a single, reusable set of component styles, and a working photograph system for vendors and menu items that degrades gracefully when a photo has not yet been supplied.

The finished system was tested by hand across all three roles rather than through an automated test suite, and that testing produced two genuinely useful outcomes beyond simply confirming the feature list worked. First, it surfaced a real defect — a CDN-dependent styling approach that failed silently in the actual browser used for testing — that would not have been caught by reading the code alone, and the fix (compiling the stylesheet locally) is now part of the delivered system. Second, because a real student account registered and placed a real order on the running instance partway through development, the project had to solve, in practice, the problem of adding new seed content (five additional vendors) without destroying live data, which produced the non-destructive `data/add-new-vendors.js` script described in Section 4.4.

5.2 Conclusion

Within the scope it was built for — ahead-of-time ordering and pickup, not delivery, and no live payment processing — CampusBites works. Registration, role-based login, vendor discovery, menu browsing, cart building, order placement with a price snapshot, live-updating order tracking, vendor order management, and administrator vendor vetting are all implemented and were confirmed working against real data rather than only against test

fixtures, as shown by the two live orders discussed in Section 4.8. The interface redesign also met its aim of presenting all three roles with one consistent visual language instead of the ad hoc styling the project started with.

The project also demonstrates, in a small and concrete way, why testing against a real running system matters more than reviewing code in isolation: the CDN styling failure was invisible in the source files and only became obvious once the page was actually opened in a browser. That single finding shaped a permanent architectural decision — compiling CSS locally rather than depending on a runtime CDN — and is arguably as significant a result of this project as any individual feature.

The system's honest limitations should be stated alongside its successes rather than left implicit. There is no automated test coverage, no real payment gateway, no delivery logistics, no order cancellation path, and — at the time of writing — no photographs yet for any of the nine vendor listings. None of these were silently dropped; each was a deliberate scope decision recorded in this report, and each is a reasonable candidate for future work.

5.3 Recommendations

For anyone extending or deploying this system beyond its current demo scope, the following are recommended:

1. Populate the nine outstanding vendor photographs before any public demonstration of the vendor listing page, since the fallback icon, while functional, is visibly a placeholder.
2. Introduce an automated test suite (for example, a small set of Jest or Supertest integration tests against the Express routes) before adding further features, since the project currently has no automated regression protection at all — every change is presently verified by hand.
3. Replace the default in-memory Express session store with a persistent store (such as Redis or a database-backed store) before running more than one server process, since the in-memory store used during development does not share sessions across processes and is explicitly unsuitable for that use in Express's own documentation.
4. If usage grows beyond a single-campus demo, migrate the JSON file store to a real database engine; the flat-file store was an appropriate, low-friction choice for this project's scale, but it has no protection against concurrent write conflicts under real multi-user load.
5. Continue to avoid inventing data — ratings, delivery-time estimates, or review counts — ahead of the features that would genuinely produce them, as was deliberately

avoided throughout this build; fabricated numbers attached to real vendor names would misrepresent those vendors.

5.4 Contributions to Knowledge

This project’s contribution is practical rather than theoretical, and is best summarised as two reusable techniques that arose directly from problems encountered while building it. The first is the non-destructive seeding pattern demonstrated in `data/add-new-vendors.js`: rather than treating “seed data” as something only ever applied to an empty database, the project separates a full-reset seed script from an idempotent, catalogue-driven “add what’s missing” script, which allowed new demo content to be added to a system that already held one genuine user’s real data without disturbing it. The second is the name-derived, extension-chaining image resolution scheme described in Section 4.5, which lets a lightweight, database-free asset (a JPEG dropped into a folder) be associated with a data record purely by a deterministic slug of its name, automatically shares one photograph across every vendor selling an identically named dish, and fails gracefully to a placeholder rather than a broken image when a photo has not yet been supplied. Both patterns are directly transferable to other small, file-backed web projects with a similar seed-data and asset-management shape.

5.5 Suggestions for Future Work

The following extensions are suggested, in roughly the order they would add the most value:

1. **Vendor-editable menus.** Menu items are currently seed data only; a vendor cannot add, edit, or remove a menu item through the interface. A vendor-facing menu management form, guarded by the same `requireRole('vendor')` middleware already used elsewhere, would be a natural next feature.
2. **Order cancellation.** As shown in the activity diagram in Figure 3.4, an order that a vendor never acts on has no defined way to be cancelled by the student who placed it. A cancellation action, allowed only while an order is still “pending,” would close that gap.
3. **Real-time updates over polling.** The four-second polling interval used for order tracking and the vendor dashboard is simple and was an explicit, documented scope decision, but a WebSocket or Server-Sent Events channel would reduce both latency and unnecessary request volume.
4. **Automated testing.** As recommended in Section 5.3, introducing unit tests for the persistence helpers (`load`, `save`, `tx`, `slugify`) and integration tests for each route would let future changes be verified automatically instead of only by hand.

5. **Ratings and reviews.** Deliberately excluded from this project's scope (see the scope decision recorded in SPEC.md), a genuine ratings system — built on actual completed orders rather than invented figures — would add real value to the vendor listing page without the risk of misrepresenting any vendor.
6. **Payment integration.** Every order is currently “pay on pickup”; integrating a real payment gateway would be a substantial addition and would need to be weighed against whether the campus vendors this system serves are equipped to accept digital payment at all.